

银行中后台上线项目完整规划方案

FinSight Bank 中后台风控运营平台

目标：用 React + TypeScript + Vite + Ant Design 构建一个可上线、可演示、可写进简历、能支撑银行开发岗面试深挖的中后台项目。

项目定位：银行信贷审批与风控运营场景。

核心能力：登录鉴权、RBAC 权限、动态菜单、客户管理、贷款申请、风控审批、操作日志、数据看板、AI 风险摘要。

上线目标：提供线上预览链接和 GitHub 源码链接，便于投递简历和面试展示。

开发方式：后续可使用 Codex 辅助生成模块代码，但架构、数据流、权限、异常链路必须自己掌握。

目录

Placeholder for table of contents	0
-----------------------------------	---

一、项目定位与简历价值

项目建议命名为 FinSight Bank 中后台风控运营平台。它不是普通 CRUD 后台，而是围绕银行信贷业务构建的中后台系统。银行、央国企、金融科技类开发岗非常看重稳定性、权限、表格、流程、审计和工程规范，因此这个项目比音乐播放器更贴近目标岗位。

项目一句话描述

面向银行信贷审批与运营管理场景的中后台系统，包含登录鉴权、RBAC 权限控制、客户管理、贷款申请管理、风控审批、数据看板、操作日志与 AI 风险摘要等模块，支持分页筛选、动态菜单、按钮权限、审批流转与异常兜底。

为什么适合银行投递

- 业务贴合：客户、贷款、授信、风控、审批、日志都是银行后台常见场景。
- 技术贴合：React、TypeScript、Ant Design、Axios、Zustand、ECharts 都是中后台常用技术。
- 面试可讲：权限系统、请求封装、分页筛选、审批流、AI 摘要、错误处理、部署上线都能被展开。
- 简历更强：相比纯前端页面项目，它能体现中后台系统设计能力和金融业务理解。

二、技术栈与选型理由

模块	技术	选型理由
构建工具	Vite + React + TypeScript	开发启动快，适合现代 React 项目；TS 便于接口和状态建模。
UI 组件	Ant Design	企业级中后台组件齐全，表格、表单、Modal、权限后台页面效率高。
路由	React Router	适合做路由守卫、动态路由和 403/404 页面。
状态管理	Zustand + persist	轻量、模板代码少，可持久化登录态、主题、菜单状态。
请求层	Axios	便于封装拦截器、token 注入、错误处理、泛型响应模型。
图表	ECharts	用于 Dashboard 风险分布、申请趋势、审批通过率等图表。
Mock	本地 Mock / json-server / MSW	第一版不依赖真实后端，先保证完整交互和上线。
部署	Vercel / Netlify / Cloudflare Pages	适合快速部署前端静态站点，生成可投递链接。

资料依据：Vite 官方指南说明支持 JavaScript/TypeScript 模板并提供现代前端开发体验；React 官方 lazy 文档说明 lazy 组件加载时会 suspend，需要 Suspense 展示加载态；Zustand 官方 persist 文档说明 persist 可以将状态保存到 storage；Ant Design 官方文档定位为用于构建丰富交互界面的 React UI 组件库。

三、系统模块总览

模块	核心功能	面试可讲点
登录鉴权	登录页、token、退出、刷新恢复	Zustand persist、路由守卫、401 处理
RBAC 权限	角色、权限码、动态菜单、按钮权限	路由权限、按钮权限、Permission 组件
Dashboard	指标卡、趋势图、风险分布图	ECharts 封装、loading/empty/error
客户管理	客户列表、筛选、详情、新增编辑	分页、搜索防抖、表单校验、脱敏
贷款申请	申请列表、状态流转、详情	复杂表格、多条件筛选、状态 Tag
风控审批	风险评分、审批通过/拒绝、AI 摘要	审批流、乐观更新、失败回滚、AI 兜底
操作日志	关键操作留痕、筛选、导出	银行审计意识、日志模型设计
系统管理	用户、角色、权限配置	中后台常见系统能力

四、核心业务流程

1. 登录到后台

用户输入账号密码，前端做表单校验。

请求登录接口，成功后保存 token、userInfo、roles、permissions。

Zustand persist 只持久化 token、userInfo、主题等长期状态。

进入后台后根据权限生成动态菜单。

后续请求在 Axios 请求拦截器中自动携带 token。

2. 贷款申请审批

审批人员进入贷款申请列表，通过客户名、状态、产品类型、风险等级进行筛选。

点击详情进入审批页，查看客户基础信息、申请金额、期限、风险评分和历史记录。

系统生成 AI 风险摘要，辅助审批人员理解风险点。

审批人员选择通过或拒绝，填写审批备注。

操作成功后更新申请状态，同时写入操作日志。失败时 Toast 提示并保留原状态。

3. AI 风险摘要链路

输入：客户画像、贷款信息、收入负债、历史逾期、风险评分。

状态：loading、result、error、empty。

请求：构建 prompt 上下文，请求 AI 摘要接口或 Mock AI 接口。

校验：检查 summary、suggestion、tags 是否存在。

兜底：AI 失败或空结果时降级为规则摘要。

定位：AI 只做辅助分析，不替代人工审批。

五、RBAC 权限系统设计

RBAC 即基于角色的权限控制：用户拥有角色，角色绑定权限，权限控制菜单、路由、按钮和接口。前端负责展示控制和交互体验，真正安全校验必须由后端完成。

角色设计

角色	说明	典型权限
admin	系统管理员	全部权限、用户管理、角色管理
risk_officer	风控专员	贷款查看、风控审批、风险报告
operator	运营人员	客户管理、贷款录入、基础查看
auditor	审计人员	操作日志查看、只读审计
viewer	只读用户	Dashboard 和部分列表查看

权限码示例

```
dashboard:view
customer:view
customer:create
customer:edit
loan:view
loan:approve
loan:reject
risk:view
log:view
system:user
system:role
```

实现点

- 路由 meta 中配置 permission 字段。
- 登录后根据 permissions 过滤可访问路由和侧边栏菜单。
- 封装 AuthRoute：未登录跳 /login，无权限跳 /403。
- 封装 Permission 组件和 usePermission hook 控制按钮显示。
- Axios 响应拦截器处理 401/403，分别跳登录或 403。

```
function Permission({ code, children }) {
  const permissions = useUserStore((state) => state.permissions)
  if (!permissions.includes(code)) return null
  return children
}
```

六、页面与路由规划

```
/login
/
/dashboard
/customers
/customers/:id
/loans
/loans/:id
/risk-review
/risk-review/:id
/operation-logs
/system/users
/system/roles
```

/403
/404

第一版优先完成登录、Dashboard、客户管理、贷款管理、风控审批、操作日志。系统管理可以做轻量版本，主要用于展示角色和权限结构。

七、目录结构设计

```
src
├── api
│   ├── request.ts
│   ├── auth.ts
│   ├── customer.ts
│   ├── loan.ts
│   ├── risk.ts
│   └── log.ts
├── components
│   ├── Permission
│   ├── PageContainer
│   ├── StatusTag
│   ├── SearchForm
│   └── RiskScore
├── layouts
│   └── BasicLayout.tsx
├── pages
│   ├── Login
│   ├── Dashboard
│   ├── Customer
│   ├── Loan
│   ├── RiskReview
│   ├── OperationLog
│   ├── System
│   ├── Forbidden
│   └── NotFound
├── router
│   ├── routes.tsx
│   └── AuthRoute.tsx
├── store
│   ├── userStore.ts
│   ├── appStore.ts
│   └── permissionStore.ts
├── types
│   ├── user.ts
│   ├── customer.ts
│   ├── loan.ts
│   ├── risk.ts
│   └── api.ts
├── utils
│   ├── format.ts
│   ├── permission.ts
│   ├── mock.ts
│   └── cache.ts
├── App.tsx
└── main.tsx
```

八、TypeScript 数据模型

ApiResponse

```
interface ApiResponse<T> {
  code: number
  message: string
  data: T
}
```

```

}

interface PageResult<T> {
    list: T[]
    total: number
    page: number
    pageSize: number
}

```

User

```

type RoleCode = 'admin' | 'risk_officer' | 'operator' | 'auditor' | 'viewer'

interface User {
    id: number
    username: string
    nickname: string
    role: RoleCode
    permissions: string[]
}

```

Customer

```

interface Customer {
    id: number
    name: string
    phone: string
    age: number
    job: string
    monthlyIncome: number
    creditLevel: 'A' | 'B' | 'C' | 'D'
    riskTags: string[]
    createdAt: string
}

```

LoanApplication

```

interface LoanApplication {
    id: number
    customerId: number
    customerName: string
    productType: '消费贷' | '经营贷' | '房贷' | '车贷'
    amount: number
    term: number
    status: 'pending' | 'risk_review' | 'approved' | 'rejected' | 'released'
    riskScore: number
    submitTime: string
    reviewer?: string
}

```

RiskReport

```

interface RiskReport {
    applicationId: number
    score: number
    level: 'low' | 'medium' | 'high'
    tags: string[]
    summary: string
    suggestion: string
}

```

九、从 0 到上线的完整构建流程

Step 1: 创建项目

```
npm create vite@latest finsight-bank-admin -- --template react-ts
cd finsight-bank-admin
npm install
npm run dev
```

Step 2: 安装依赖

```
npm install react-router-dom zustand axios antd @ant-design/icons echarts echarts-for-react dayjs
npm install -D eslint prettier
```

Step 3: 搭建基础框架

配置 `main.tsx` 和 `App.tsx`。

配置 React Router 路由表。

实现 Login 页面和 BasicLayout。

实现 AuthRoute 路由守卫。

实现 `userStore`、`permissionStore`。

Step 4: 封装请求层

创建 `api/request.ts`。

请求拦截器注入 `token`。

响应拦截器统一处理 `code`、`401`、`403`、`5xx`。

封装泛型 `request<T>`。

业务 API 按模块拆分。

Step 5: 准备 Mock 数据

先用本地数组模拟 `customers`、`loans`、`riskReports`、`logs`。

用 `Promise` + `setTimeout` 模拟接口延迟。

实现筛选、分页、状态更新、审批操作。

后续可替换为 `json-server` 或真实后端。

Step 6: 完成核心页面

Dashboard: 统计卡片 + 图表。

客户管理: 列表、筛选、分页、详情、编辑。

贷款管理: 列表、状态筛选、审批入口。

风控审批: 风险评分、AI 摘要、通过/拒绝。

操作日志: 审批操作和权限操作留痕。

Step 7: 加入工程优化

`React.lazy` + `Suspense` 做路由懒加载。

搜索筛选做 `debounce`。

表格做分页加载。

请求失败用 Toast + retry + fallback。

相同参数可做 cacheKey 缓存。

权限按钮用 Permission 组件统一控制。

Step 8：构建并部署

npm run build

npm run preview

GitHub 新建仓库并推送代码。

Vercel 导入 GitHub 仓库。

Build Command: npm run build。

Output Directory: dist。

生成线上项目链接，写入简历。

十、开发排期建议

阶段	目标	产出
第 1 周	项目骨架、登录、路由、布局、权限 store	能登录并进入后台，动态菜单可用
第 2 周	Dashboard、客户管理、贷款列表	中后台基础页面成型
第 3 周	风控审批、状态流转、操作日志、按钮权限	银行业务主链路完整
第 4 周	AI 风险摘要、兜底、重新生成、规则降级	形成项目亮点
第 5 周	工程优化、懒加载、防抖、缓存、错误处理	面试可讲工程细节
第 6 周	部署上线、README、演示录屏、简历包装	可投递、可演示、可复盘

如果时间紧，可以压缩为 3 周：第 1 周完成骨架和权限，第 2 周完成核心业务页面，第 3 周完成 AI 摘要、工程优化和上线部署。

十一、MVP 优先级

第一版不要贪大。先完成能上线、能演示、能讲清楚的主链路。

必须完成

登录和退出。

动态菜单和路由守卫。

Dashboard 数据看板。

客户列表和贷款列表。

贷款审批详情页。

通过/拒绝审批操作。

操作日志记录。

AI 风险摘要和兜底。

Vercel 上线。

可后续增强

用户管理和角色配置。

复杂图表和大屏模式。

虚拟列表。

Web Worker 处理复杂筛选。

Cypress/Playwright 测试。

真实大模型 API 接入。

十二、简历包装文案

项目名称

FinSight Bank 中后台风控运营平台 | React + TypeScript + Vite + Ant Design + Zustand

项目描述

面向银行信贷审批与运营管理场景的中后台系统，包含登录鉴权、RBAC 权限控制、客户管理、贷款申请管理、风控审批、数据看板、操作日志与 AI 风险摘要等模块，支持分页筛选、动态菜单、按钮权限、审批流转与异常兜底。

简历亮点

基于 RBAC 设计路由权限、动态菜单和按钮级权限，封装 Permission 组件与 usePermission hook，实现不同角色访问控制。

封装 Axios 请求层和 TypeScript 泛型响应模型，统一处理 token 注入、错误提示、401/403、loading 和分页数据结构。

实现客户管理与贷款申请管理模块，支持多条件筛选、分页加载、详情查看、审批流转和操作日志记录。

使用 Zustand 管理用户态、权限态和应用配置，并通过 persist 持久化 token、用户信息和主题配置，避免刷新后状态丢失。

接入 AI 风险摘要模块，根据客户画像、贷款申请、风险评分生成审批摘要和风险建议，并设计 loading、空结果兜底、规则降级等异常链路。

使用 React.lazy + Suspense 做路由级懒加载，结合搜索防抖、分页加载和缓存策略优化中后台性能体验。

十三、面试讲解主线

面试时不要把项目讲成页面堆叠，而要讲成三条主线：中后台基础能力、银行业务流程、工程增强能力。

推荐开场

这个项目是一个面向银行信贷审批场景的中后台系统。我把它分成三条主线：第一条是中后台基础能力，包括登录鉴权、RBAC 权限、动态菜单、路由守卫和按钮权限；第二条是业务流程，包括客户管理、贷款申请、风

控审批和操作日志；第三条是工程增强，包括 Axios 请求封装、Zustand 状态管理、分页筛选、路由懒加载和 AI 风险摘要。

容易被深挖的问题

RBAC 权限系统怎么设计？

动态菜单和路由权限怎么做？

按钮权限怎么封装？

Axios 请求层怎么封装？

分页筛选怎么做？

审批通过/拒绝的状态流转怎么处理？

AI 风险摘要输入是什么？怎么兜底？

Zustand persist 存哪些状态，不存哪些状态？

如何避免重复请求和旧请求覆盖新结果？

如何部署上线并配置生产环境？

十四、Codex 辅助开发使用建议

你接下来会用 Codex 辅助开发，但不要让它随意生成整套系统。推荐按模块逐步下指令，自己把控架构。

推荐提示词模板

请在当前 React + TypeScript + Vite + Ant Design 项目中，按照已有目录结构，帮我实现 [模块名]。

要求：

1. 使用 TypeScript 明确类型；
2. 使用 Ant Design 组件；
3. 接入 src/api 中的请求函数；
4. loading、empty、error 状态完整；
5. 代码风格和现有项目保持一致；
6. 不要修改无关文件；
7. 给出关键实现说明。

开发顺序

先让 Codex 生成 types 和 mock 数据。

再生成 api 模块。

再生成 store。

再生成页面组件。

最后做联调、权限、异常和样式优化。

每完成一个模块就运行项目，避免堆积错误。

核心原则：Codex 可以辅助写代码，但项目架构、状态流、权限逻辑、请求链路和面试话术必须由你掌握。

十五、参考资料

Vite 官方指南：支持通过 create vite 创建 JavaScript/TypeScript 模板，适合现代前端项目。

React lazy 官方文档: lazy 组件加载时会 suspend, 需要 Suspense 展示 fallback。

Zustand persist 官方文档: persist 可将状态存储到 localStorage 等 storage。

Ant Design 官方文档: React UI 组件库, 适合构建丰富交互界面和中后台应用。